

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 4. Operators and Expressions

Lecture: 34. Arithmetic Assignment Operators

Section 1: Lecture Summary

This lecture covers the topic of **arithmetic assignment operators** in Python, which combine arithmetic operations with assignment to simplify code. The instructor begins by reviewing traditional arithmetic operators and introduces how their combined forms, known as arithmetic assignment operators (e.g., `+=`, `-=`, `=`, `/=`, etc.), function. The main focus is on explaining how these operators update the value of the variable on the left side by applying an operation with the right-side operand and storing the result back into the same variable. Detailed explanations show how `a = a + 1` can be simplified to `a += 1`, and this pattern extends to all arithmetic operations.

The instructor demonstrates the practical use of these operators with various examples involving variables `a` and `b`, such as addition assignment, subtraction assignment, multiplication assignment, division assignment, modulus assignment, and exponentiation assignment. Emphasis is placed on how these operators are memory efficient (creating new values rather than modifying in place) and useful in loops for counting and updating counters. The lecture concludes by encouraging students to practice these operators, noting that they will be more commonly used in upcoming lessons involving loops.

Section 2: Key Concepts and Explanations

- Arithmetic Operators Refresher:

- Symbols like `+`, `-`, `*`, `/`, `//`, `%`, `**` represent arithmetic operations: addition, subtraction, multiplication, division (float), floor division, modulo, and exponentiation, respectively.

- Arithmetic Assignment Operators:

- These operators perform an arithmetic operation on the variable and assign the result back to it.

- General form: ``variable <operator>= value``

- Examples:

- ``a += b`` is equivalent to ``a = a + b``

- ``a -= b`` is equivalent to ``a = a - b``

- ``a = b`` is equivalent to ``a = a b``

- ``a /= b`` is equivalent to ``a = a / b``

- ``a //= b`` is equivalent to ``a = a // b`` (floor division)

- ``a %= b`` is equivalent to ``a = a % b`` (modulus)

- ``a = b`` is equivalent to ``a = a b`` (power)

- Variable Memory Model:

- When ``a = a + 1`` is executed, Python does not modify the original value in memory but creates a new memory space for the updated value.

- Arithmetic assignment operators simplify the notation but functionally perform the same operations.

- Usage in Expressions:

- Arithmetic assignment can be used with simple values or complex expressions.

- For example:

```
x += a * b - c
```

This means evaluate ``a b - c``, add to current ``x``, and store back into ``x``.

- Common Use Cases:

- Incrementing counters in loops (``a += 1``).

- Modifying variables concisely without repeating variable names.

- Improving code readability and reducing redundancy.

Section 3: Example Code and Use Cases

Incrementing a variable

```
a = 1
a = a + 1  # traditional way
print(a)   # Output: 2

a += 1     # arithmetic assignment operator
print(a)   # Output: 3

a += 1
print(a)   # Output: 4
```

Explanation: Shows how `a += 1` increments `a` by 1, simulating a counter.

Using variables with arithmetic assignment

```
a = 6
b = 3

a = a + b      # addition assignment traditional
print(a)       # Output: 9

a = 6          # reset a
a += b         # addition assignment operator
print(a)       # Output: 9

a = 6
a -= b         # subtraction assignment
print(a)       # Output: 3

a = 6
a *= b         # multiplication assignment
print(a)       # Output: 18

a = 6
a /= b         # division assignment (float division)
print(a)       # Output: 2.0
```

```

a = 6
b = 5
a %= b          # modulus assignment
print(a)        # Output: 1

a = 2
b = 3
a **= b         # exponentiation assignment
print(a)        # Output: 8 (2 to the power 3)

```

Explanation: Demonstrates arithmetic assignment operators for addition, subtraction, multiplication, division, modulus, and exponentiation with two variables.

Complex expression with arithmetic assignment

```

a = 6
b = 3
c = 2
x = 4

x = x + a * b - c          # traditional expression
print(x)                  # Output: 20

x = 4                     # reset value of x
x += a * b - c            # arithmetic assignment with expression
print(x)                  # Output: 20

```

Explanation: Shows using arithmetic assignment with a compound expression, simplifying the code without changing the result.

Section 4: Key Takeaways

- Arithmetic assignment operators perform arithmetic operations combined with assignment in a concise form.
- Operators include `+=`, `-=`, `\*=`, `/=`, `//=`, `%=` and `\*\*=`.
- Using these operators avoids redundancy of repeating variables on both sides of the assignment.
- Python creates new memory for results; it doesn't modify original values in place.

- These operators are especially useful for incrementing counters and updating variables inside loops.
- Complex expressions can also be used on the right side of arithmetic assignment operators.
- Mastery of these operators will simplify your Python code and improve readability.